

Binary GCD on Unsigned 64-Bit Integers

LeanExe library PoC

September 2026

1 Task and interface

`gcdBinary` computes the greatest common divisor of two `UInt64` values using shifts, comparisons, and subtraction. It accepts every pair of input words. The intended result is $\text{gcd}(a, b)$, with $\text{gcd}(0, 0) = 0$.

LeanExe compiles the Lean function and its callers to WebAssembly (WASM). The examples run that WASM in Wasmtime.

2 Verification status

Lean checks the source types. Execution tests pass for selected inputs: both GCD implementations agree with an arbitrary-precision integer reference in WASM and native Lean. The native test also checks Lean's `Nat.gcd`. Formal correctness and termination proofs for this source function, its generated WASM, and its fraction client remain deferred.

3 Algorithm and executable source

Common factors of two can be removed from both operands and restored at the end. A factor of two in just one operand can be removed when the other is odd. Subtracting the smaller odd operand from the larger produces an even difference. These identities are described in NIST's binary GCD entry [1].

```
namespace LeanExe.Lib.NumberTheory

def gcdBinary (a b : UInt64) : UInt64 := Id.run do
  if a == 0 then return b
  if b == 0 then return a
  let mut x := a
  let mut y := b
  let mut shift : UInt64 := 0
  while (x ||| y) &&& 1 == 0 do
    x := x >>> 1
    y := y >>> 1
    shift := shift + 1
  while x &&& 1 == 0 do
    x := x >>> 1
  while y != 0 do
    while y &&& 1 == 0 do
      y := y >>> 1
```

```

    if x > y then
      let saved := x
      x := y
      y := saved
    y := y - x
    return x <<< shift
end LeanExe.Lib.NumberTheory

```

The listing comes from `Basic.lean`. Zero inputs return before a shift loop can encounter zero. The common shift count is at most 63 for nonzero inputs. Ordering the operands before subtraction gives a nonnegative difference. The result fits in `UInt64`. The loop keeps two operands and a shift count.

4 A shared client task

The example program provides two fraction-reduction functions. One calls Euclidean GCD and the other calls binary GCD. Both use the same result helper, which rejects a zero denominator and divides the numerator and denominator by the selected divisor. The commands run from the repository root after following the setup guide.

```

tools/seminum gcd-binary 48 18
6
tools/seminum fraction-binary 48 18
8/3
tools/seminum fraction 48 18
8/3

```

Arguments are unsigned decimal words through $2^{64} - 1$. Invalid syntax and a zero denominator produce status 2 and a diagnostic on standard error.

5 Tests

From the repository root, `node test/seminum.js` runs the comparisons described above on the same word pairs for both GCD implementations. Inputs include zero, maximum words, powers of two, consecutive large Fibonacci numbers, and generated full-width pairs. Fraction cases check reduction and rejection of a zero denominator. CLI tests check results and invalid-input diagnostics.

The test source is `test/seminum.js`. The native reference driver is `test/SeminumNative.lean`.

References

- [1] Paul E. Black, “binary GCD,” NIST Dictionary of Algorithms and Data Structures, 2020. <https://xlinux.nist.gov/dads/HTML/binaryGCD.html>. This entry supplies the parity identities and identifies Knuth’s Algorithm 4.5.2B. The implementation here counts shifts and handles zero inputs before the loop.